Compiler Design Course Project

Compiler for a subset of C++

💡 Tools to use: Flex, Yacc, and .c files

Features to include

- int, float
- for, while
- if-else
- +, -, *, /
- <, >, =
- arrays
- functions
- single and multiline comments
- Nested if-else
- Preprocessor directives
- Function definition
- variable definition and declaration
- assignment operation
- arithmetic operations

To identify

- Header
- Keywords
- Main function
- Special symbols
- Identifiers

- Operator
- Integer constant
- Float constant
- Predefined functions (eg: printf())
- Type specifier (eg: %d)

Set

Keywords

int, float, if, else, for, while, return

Operators

+, -, *, /, <, >, =

Special Symbols

, ; ( ) { } [ ]

Constants

- Integer: [0-9]+
- Float: [0-9]+(.)[0-9]+

Comments

- Single line: //
- Multi line: /* */

Preprocessor directives

#include, #define

Type checking

- Float should not be assigned to int type
- Return type of a function should match the defined data type

Variable rules

- Variables must be declared before using them

- Redeclaration of variables is not allowed

Array

- While accessing elements, the index must be an integer

User defined functions rules

- Number of parameters must match
- Data type of parameters must match
- Return type must be the same as defined one

Phases

Lexical Analysis

Reads the input .c file character wise and groups them into meaningful tokens. Removes whitespaces and comments, while recognizing keywords, identifiers, constants, operators, and preprocessor directives. Output will be tokens which are passed to the Syntax Analyzer.

Syntax Analyzer

Analyses tokens from the lexical analyzer and verifies whether it follows the grammar rules of C. Constructs a parse tree representing the input program. If a syntax error is found, it reports the line number at which the error occurs.

Semantic Analysis

The semantic analyzer verifies type correctness, variable declarations, and scope rules. It builds and maintains a symbol table to track identifiers and their attributes. It reports semantic errors like undeclared variables, or type mismatches.

Sample Program

```c
#include <stdio.h>
#define LIMIT 5

// Sample comments
/* sample
testing 1,2,3
comment */
```

```
class Student {
    int rollNo;
    float marks;
    int grade;
    int age;

    Student(int r, float m) {
        rollNo = r;
        marks = m;
        grade = 0;
        age = 18;
    }

    int getMarks() {
        return marks;
    }

    float computeGrade(int count) {
        float g = marks / count;
        return g;
    }

    void display() {
        printf("%d", rollNo);
        printf("%d", age);
    }
};

int multiply(int a, int b) {
    int result = a * b;
    return result;
}

int main() {
    int x = 5;
    float y = 2.5;
    int z = 9.99;
```

```
    int marks[LIMIT];
    int i;
    for (i = 0; i < LIMIT; i++) {
        marks[i] = i * 10;
    }
    float idx = 1.5;
    int val = marks[idx];
    if (x > 3) {
        if (x > 6) {
            z = z + 1;
        } else {
            z = z - 1;
        }
    } else {
        z = 0;
    }
    int j = 0;
    while (j < LIMIT) {
        marks[j] = marks[j] + x;
        j = j + 1;
    }
    int total = x + undeclaredVar;
    Student s(101, 88.5);
    s.display();
    int m = s.getMarks();
    int x = 100;
    int result = multiply(x, 2);
    printf("%d", result);
    return 0;
}
```

Semantic Error at line 25: return type mismatch: function declared as 'int' but returns 'float'
Semantic Error at line 53: cannot assign float value to int variable 'z'
Semantic Error at line 63: array index must be an integer
Semantic Error at line 72: undeclared variable 'undeclaredVar'
Semantic Error at line 80: variable 'x' already declared in this scope

Errors: 5
Program not accepted.

Sample

```
#include <stdio.h>
#define LIMIT 5

// Sample comments
/* sample
testing 1,2,3
comment */

class Student {
    int rollNo;
    float marks;
    int grade;
    int age;

    Student(int r, float m) {
        rollNo = r;
        marks = m;
        grade = 0;
        age = 18;
    }

    int getMarks() {
        return marks;
    }

    float computeGrade(int count) {
        int g = marks / count;
        return g;
    }

    void display() {
        printf("%d", rollNo);
        printf("%d", age);
```

```
    }
};

int multiply(int a, int b) {
    int result = a * b;
    return result;
}

int main() {
    int x = 5;
    float y = 2.5;
    int z = 9;
    int marks[LIMIT];
    int i;
    for (i = 0; i < LIMIT; i++) {
        marks[i] = i * 10;
    }
    float idx = 1;
    int val = marks[idx];
    if (x > 3) {
        if (x > 6) {
            z = z + 1;
        } else {
            z = z - 1;
        }
    } else {
        z = 0;
    }
    int j = 0;
    while (j < LIMIT) {
        marks[j] = marks[j] + x;
        j = j + 1;
    }
    int total = x;
    Student s(101, 88.5);
    s.display();
    int m = s.getMarks();
    int r= multiply(x, 2);
```

```
    printf("%d", result);
    return 0;
}
```